Задача A1. Как построить минимальный остов?

Жданок Дарья БПИ248

```
ALG_1(G):
  отсортировать ребра грава G
  в порядке невозрастания весов;
  T = E;
  foreach (e ∈ E в порядке невозрастания весов)
    if (ребра T - {e} образуют связный граф)
      T = T - {e};
  return T;
```

```
ALG_2(G):
  T = ∅;
  foreach (e ∈ E, выбранное случайным образом)
    if (ребра T ∪ {e} образуют граф без циклов)
      T = T ∪ {e};
  return T;
```

```
ALG_3(G):
  T = ∅;
  foreach (e ∈ E, выбранное случайным образом)
    T = T ∪ {e};
    if (в T имеется цикл из ребер c ⊆ T)
      e_max = ребро с максимальным весом
              в цикле c;
      T = T - {e_max};
  return T;
```

Ссылка на репозиторий с кодом: https://github.com/gogogok/set6

1) ALG1

1) Наиболее эффективная по временной сложности реализация

Самая быстрая реализация по времени на практике и по асимптотике – построить MST через Краскала

- Используем DSU (Union-Find)
- Сортируем рёбра по неубыванию (от меньших к большим) и добавляем ребро, если оно соединяет разные компоненты

ALG1 гарантированно строит MST, значит самая эффективная реализация вычисления MST – Краскал или Прим

2) Обоснование оценки сложности

- Сортировка рёбер: O(m Log m)
- DSU-операции: m раз find/union -> O(m * a(n)) (почти линейно)
- Итого: O(m log m)

3) Формируется ли MST в T?

Да, формируется минимальное остовное дерево

Обоснование: при рассмотрении ребра e (сейчас мы идём от больших весов к меньшим):

- если после удаления e граф остаётся связным, значит есть альтернативный путь между его концами -> e лежит в некотором цикле
- так как мы идём от больших к меньшим, e -> одно из самых тяжёлых рёбер в текущем цикле, и самое тяжёлое ребро цикла не нужно MST -> его можно выкинуть

Пример

Пусть есть вершины A B и C

Связи и веса:
AB=1, BC=2, AC=3
Удаляем 3 (граф остаётся связным) -> остаются 1 и 2 -> это MST.

2) ALG2

1) Наиболее эффективная реализация

Используем DSU для проверки цикла. Если find(u) != find(v), добавляем, union(u,v). Иначе пропускаем

Почему это эффективно?

1. Нам обязательно нужно проверить связность концов ребра

Без этого невозможно понять, образуется ли цикл.

2. Проверка связности в динамически растущем лесу

типичная задача DSU

3. Быстрее, чем O(m), быть не может

Мы обязаны хотя бы один раз посмотреть на каждое из m рёбер

2) Обоснование оценки сложности

- Перебор всех рёбер: m
- DSU:O(a(n)) на операцию
  Итого: O(m * a(n)) (почти O(m))

3) Формируется ли MST в T?

Нет, вообще не гарантируется, потому что алгоритм не смотрит на веса

Контрпример:

AB=1, BC=2, AC=100
Если порядок рёбер: AC(100), BC(2), AB(1)

- берём AC и BC → уже дерево
- AB создаёт цикл → не берём
  Итог вес 102, а MST вес 3.

3) ALG3

1) Наиболее эффективная по времени реализация и обоснование

На каждом шаге T – это лес (потому что как только появляется цикл, мы его разрушаем).

Если мы добавили ребро (u,v) и образовался цикл, то цикл = путь от u до v в текущем дереве + новое ребро

Значит нам нужно:

1. Проверить, связаны ли u и v
2. Найти максимум веса на пути u -> v
3. Если новое ребро легче, удалить максимум

Используем:

- DSU, чтобы проверять связность
- Представление T как списка смежности
- DFS для поиска пути и максимального ребра

2) Обоснование оценки сложности

Пусть:

- n – число вершин
- m – число рёбер

DSU:

$O(m\,\alpha(n))$

Каждый раз, когда возникает цикл, мы делаем DFS

DFS в дереве:

$O(n)$

В худшем случае цикл может возникать почти для каждого ребра.

Значит:

- $O(mn)$

3) Формируется ли MST в T?

Да

В любом цикле ребро максимального веса не принадлежит ни одному минимальному остовному дереву.

ALG3 при появлении цикла удаляет максимальное ребро

Значит:

- никогда не удаляется ребро, которое обязано быть в MST
- постепенно удаляются все лишние тяжёлые рёбра
- в конце остаётся связный ацикличный граф из n -1 рёбер
- по свойству цикла это MST

Пример:

AB=1, BC=2, AC=3

Добавляем 1

Добавляем 3

Добавляем 2

цикл

max = 3

удаляем 3

Остаются 1 и 2 -> MST.